

Tugas Praktikum OOP - Modul 6

Polymorphism, Method Overriding, Method Overloading, dan Dynamic Binding

Nama : Rafael Rizky
NRP : 3125522007

Dosen : Nirwana Haidar Hari, S.Pd., M.Kom.
Kampus : PENS PSDKU Sumenep (2026)

HALAMAN 1 — ANALISIS, SPRINT PLANNING & AUDIT BEHAVIOR P6

1. **Nama Project:** Sistem Kasir Sederhana

2. **Sprint Goal:** Mengembangkan behavior object pada proyek melalui method overriding (@Override) dan polymorphism sehingga subclass dapat merespons pemanggilan method yang sama dengan perilaku berbeda, serta menyediakan fleksibilitas input pembayaran melalui method overloading.

3. Sprint Backlog (P6)

ID	Sprint Backlog	Status
SB-01	Audit hierarki Superclass dan Subclass hasil P5	DONE
SB-02	Menentukan behavior method yang dapat dioverride pada hierarki Produk	DONE
SB-03	Implementasi method overriding (@Override) pada ProdukMakanan dan ProdukElektronik	DONE
SB-04	Implementasi method overloading pada Transaksi (tambahItem dan prosesTransaksi)	DONE
SB-05	Implementasi upcasting dan polymorphic reference	DONE
SB-06	Membuat dan mengiterasi polymorphic collection (array of Superclass Produk[])	DONE
SB-07	Melakukan pengujian pembuktian dynamic binding saat program dijalankan	DONE

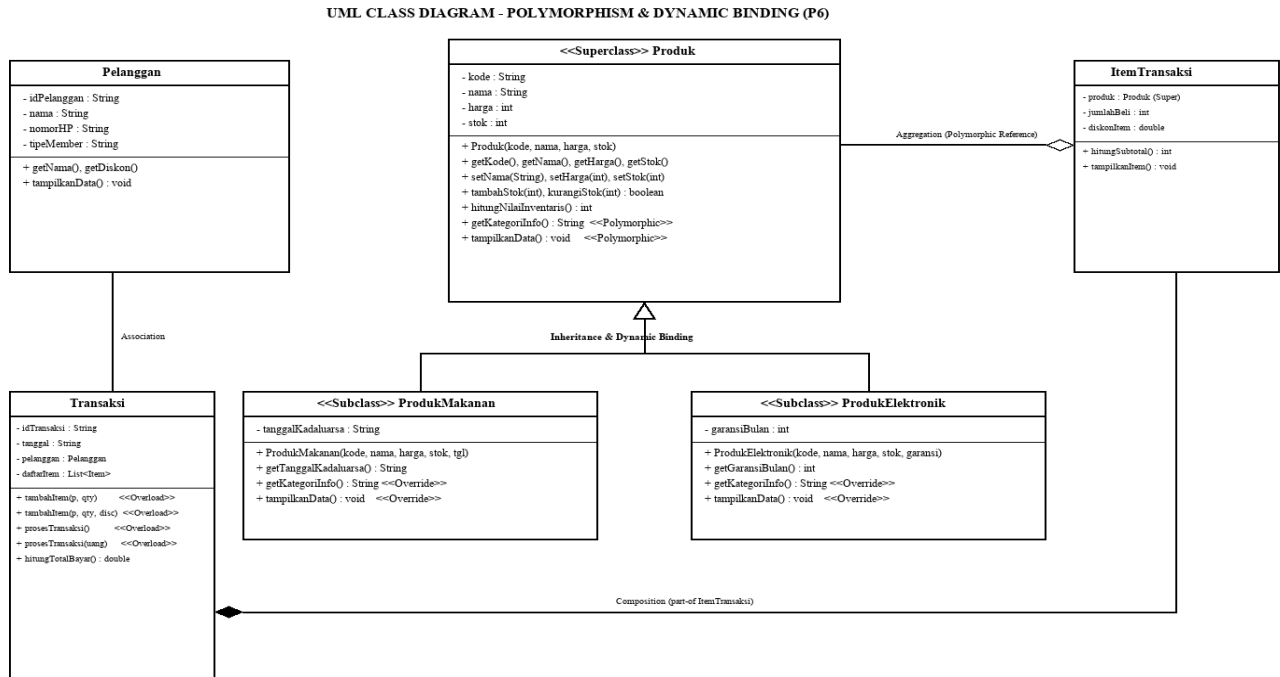
4. Hierarchy Class P5 & Behavior yang Dipilih untuk Polymorphism:

Hierarki: **Produk** (Superclass) ▲ ■■ **ProdukMakanan** & **ProdukElektronik** (Subclass).

Behavior yang dipilih untuk overriding adalah method tampilkanData() dan getKategoriInfo().

Superclass	Subclass	Method Dioverride	Perilaku Spesifik pada Subclass
Produk	ProdukMakanan	tampilkanData(), getKategoriInfo()	Kategori pangan & mencetak tanggal kadaluarsa.
Produk	ProdukElektronik	tampilkanData(), getKategoriInfo()	Kategori hardware & mencetak masa garansi unit.

5. Class Diagram Terbaru (P6)



Gambar 1. UML Class Diagram Polymorphism, Overriding, dan Overloading (P6)

6. Contoh Overriding, Overloading, dan Upcasting:

- **Contoh Overriding:** Subclass mendefinisikan ulang method superclass menggunakan @Override:
 - ProdukMakanan.tampilkanData() mencetak tanggal kadaluarsa.
 - ProdukElektronik.tampilkanData() mencetak masa garansi bulan.
- **Contoh Overloading:** Pada class Transaksi:
 - tambahItem(Produk, int) (reguler) vs tambahItem(Produk, int, double disc) (promo item).
 - prosesTransaksi() (standar) vs prosesTransaksi(double uang) (tunai & hitung kembalian).
- **Penjelasan Upcasting:** Objek subclass dirujuk menggunakan tipe referensi superclass (Produk p = new ProdukMakanan(...)). Memungkinkan polymorphic collection dan runtime dispatch.

7. Screenshot Hasil Running Program (Main.java)



Gambar 2. Bukti Running Program Main.java pada Terminal (Eksekusi Pengujian P6)

8. Tabel Pengujian Dynamic Binding:

Reference Type	Actual Object	Method Dipanggil	Perilaku / Output yang Dihasilkan
Produk (Superclass)	ProdukMakanan	tampilkanData()	Mencetak format makanan & Kadaluarsa: 25-10-2026
Produk (Superclass)	ProdukElektronik	tampilkanData()	Mencetak format elektronik & Garansi: 6 Bulan
Produk (Superclass)	ProdukMakanan	getKategoriInfo()	Output: 'Makanan & Minuman Segar (Konsumsi)'
Produk (Superclass)	ProdukElektronik	getKategoriInfo()	Output: 'Elektronik & Aksesoris Gadget (Hardware)'

9. Sprint Review & Retrospective:

Item	Hasil
Superclass & subclass tersedia	Superclass Produk serta Subclass ProdukMakanan dan ProdukElektronik tersedia.
Method overriding berhasil	Method tampilkanData() dan getKategoriInfo() berhasil dioverride dengan @Override.
Behavior berbeda terbukti	ProdukMakanan dan ProdukElektronik memiliki output spesifik sesuai domainnya.
Method overloading tersedia	tambahItem dan prosesTransaksi berhasil dioverload dengan parameter berbeda.
Upcasting & Dynamic binding	Terbukti pemanggilan method via referensi Produk mengeksekusi objek aktual di runtime.
Program berjalan	Kompilasi (javac) dan eksekusi (java) berhasil 100% bebas error.

What Went Well? Polymorphism dan dynamic binding berjalan sempurna; penambahan overloading pembayaran tunai dan diskon item memberi nilai guna nyata.

What Went Wrong? Perlu ketelitian dalam membedakan overloading (compile-time) dan overriding (runtime).

Improvement: Pada sprint berikutnya (P7: Abstraction & Interface), Superclass Produk dapat ditransformasikan menjadi Abstract Class atau mengimplementasikan Interface kontrak bisnis.